Sumcheck 프로토콜 구현 상세 분석

개요

Sumcheck 프로토콜은 영지식 증명 시스템의 핵심 구성 요소로, 다변수 다항식의 합을 효율적으로 검증하는 대화형 증명 프로토콜입니다. Proof of SQL에서는 SQL 쿼리의 정확성을 증명하기 위해 이 프로토콜을 사용합니다.

수학적 배경

Sumcheck 프로토콜의 목적

주어진 다변수 다항식 $f(x_1, x_2, ..., x_n)$ 에 대해, 다음 합이 주장된 값 H 와 같음을 증명:

$$\Sigma_{x_1 \in \{0,1\}} \Sigma_{x_2 \in \{0,1\}} ... \Sigma_{x_n \in \{0,1\}} f(x_1, x_2, ..., x_n) = H$$

프로토콜의 핵심 아이디어

1. **라운드별 축약**: 각 라운드에서 하나의 변수를 고정하여 차원을 줄임
2. **일변수 다항식**: 각 라운드에서 일변수 다항식을 전송
3. **재귀적 검증**: 마지막 라운드에서 단일 점에서의 평가값 검증

파일 구조 분석

`crates/proof-of-sql/src/proof_primitive/sumcheck/proof.rs`

SumcheckProof 구조체

```rust
#[derive(Clone, Debug, Serialize, Deserialize)]
pub struct SumcheckProof<S: Scalar> {
pub(super) coefficients: Vec<S>,
}
```

구조 분석:

- `coefficients` : 각 라운드에서 전송되는 일변수 다항식의 계수들
- **직렬화 지원**: 네트워크 전송과 저장을 위한 serde 지원
- **제네릭 스칼라**: 다양한 유한체에서 동작 가능

Subclaim 구조체

```rust
pub struct Subclaim<S: Scalar> {
pub evaluation_point: Vec<S>,
pub expected_evaluation: S,
pub max_multiplicands: usize,
}
```

구조 분석:

- `evaluation_point` : 최종 평가 지점 (각 라운드의 챌린지 값들)
- `expected_evaluation` : 해당 지점에서의 기대 평가값
- `max_multiplicands` : 다항식의 최대 곱셈 인수 개수

증명 생성 과정

`create` 메서드 분석

```
#[tracing::instrument(name = "SumcheckProof::create", level = "debug", skip_all)
pub fn create(
transcript: &mut impl Transcript,
evaluation_point: &mut [S],
mut state: ProverState<S>,
) -> Self {
log::log_memory_usage("Start");


assert_eq!(evaluation_point.len(), state.num_vars);
transcript.extend_as_be([((state.max_multiplicands + 1) * state.num_vars) as u64
// This challenge is in order to keep transcript messages grouped. (This simplif
transcript.scalar_challenge_as_be::<S>();
```

초기화 단계:

- **메모리 로깅**: 성능 모니터링을 위한 메모리 사용량 추적
- **길이 검증**: 평가 지점 배열의 길이가 변수 개수와 일치하는지 확인
- **Transcript 확장**: Fiat-Shamir 변환을 위한 메타데이터 추가
- **그룹화 챌린지**: Solidity 구현 단순화를 위한 메시지 그룹화

라운드별 증명 생성

```
let mut r = None;
let mut coefficients = Vec::with_capacity(state.max_multiplicands * state.num_va
for scalar in evaluation_point.iter_mut().take(state.num_vars) {
```

```
let round_evaluations = prove_round(&mut state, &r);

let round_coefficients =

interpolate_evaluations_to_reverse_coefficients(&round_evaluations);

transcript.extend_scalars_as_be(&round_coefficients);

coefficients.extend(round_coefficients);

*scalar = transcript.scalar_challenge_as_be();

r = Some(*scalar);

}
```

라운드 처리 과정:

1. **라운드 평가**: `prove_round` 를 통해 현재 라운드의 일변수 다항식 평가값들 계산
2. **계수 보간**: 평가값들로부터 다항식 계수 복원 (라그랑주 보간)
3. **Transcript 업데이트**: 계수들을 transcript에 추가
4. **챌린지 생성**: 검증자의 무작위 챌린지 값 생성
5. **상태 업데이트**: 다음 라운드를 위한 변수 고정

검증 과정

`verify_without_evaluation` 메서드

```
pub fn verify_without_evaluation(

&self,

transcript: &mut impl Transcript,

num_variables: usize,

claimed_sum: &S,
```

```
) -> Result<Subclaim<S>, ProofError> {
```

검증 단계:

1. **형식 검증**:

```
let coefficients_len = self.coefficients.len();
if coefficients_len % num_variables != 0 {
return Err(ProofError::VerificationError {
error: "invalid proof size",
});
}
let max_multiplicands = (coefficients_len / num_variables) - 1;
```

2. **라운드별 검증**:

```
for round_index in 0..num_variables {
let start_index = round_index * (max_multiplicands + 1);
transcript.extend_scalars_as_be(
&self.coefficients[start_index..=(start_index + max_multiplicands)],
);
let round_evaluation_point = transcript.scalar_challenge_as_be();
evaluation_point.push(round_evaluation_point);
```

3. **일관성 검증**:

```
let mut round_evaluation = self.coefficients[start_index];

let mut actual_sum =

round_evaluation + self.coefficients[start_index + max_multiplicands];

for coefficient_index in (start_index + 1)..=(start_index + max_multiplicands) {

round_evaluation *= round_evaluation_point;

round_evaluation += self.coefficients[coefficient_index];

actual_sum += self.coefficients[coefficient_index];

}

if actual_sum != expected_evaluation {

return Err(ProofError::VerificationError {

error: "round evaluation does not match claimed sum",

});

}
```

검증 로직:

- **다항식 평가**: 챌린지 지점에서 일변수 다항식 평가
- **합 검증**: `f(0) + f(1) = 이전 라운드의 기대값` 확인
- **연쇄 검증**: 각 라운드의 출력이 다음 라운드의 입력과 일치하는지 확인

ProverState 분석

상태 구조

```rust
#[derive(Debug)]
pub struct ProverState<S: Scalar> {
/// Stores the list of products that is meant to be added together. Each multipl
/// the index in `flattened_ml_extensions`
pub list_of_products: Vec<(S, Vec<usize>)>,
/// Stores a list of multilinear extensions in which `self.list_of_products` poi
pub flattened_ml_extensions: Vec<Vec<S>>,
pub num_vars: usize,
pub max_multiplicands: usize,
pub round: usize,
}
```

상태 요소 분석:

- `list_of_products` : 합산할 곱셈 항들의 목록 (계수, 인덱스 벡터)
- `flattened_ml_extensions` : 다선형 확장(Multilinear Extension)들의 평면화된 표현
- `num_vars` : 총 변수 개수
- `max_multiplicands` : 단일 곱셈 항의 최대 인수 개수
- `round` : 현재 라운드 번호

다선형 확장 (Multilinear Extension)

다선형 확장은 불린 하이퍼큐브 $\{0,1\}^n$ 에서 정의된 함수를 전체 필드로 확장하는 방법입니다:

$$MLE(x_1, \ldots, x_n) = \Sigma_{w\in\{0,1\}^n} f(w) \cdot \Pi_i (x_i w_i + (1-x_i)(1-w_i))$$

라운드 증명 과정

prove_round 함수 분석

```
#[tracing::instrument(level = "debug", skip_all)]
pub fn prove_round<S: Scalar>(prover_state: &mut ProverState<S>, r_maybe: &Optio
```

변수 고정 단계

```
if let Some(r) = r_maybe {
assert!(
prover_state.round != 0,
"first round should be prover first."
);

// fix argument
if_rayon!(
prover_state.flattened_ml_extensions.par_iter_mut(),
prover_state.flattened_ml_extensions.iter_mut()
)
.for_each(|multiplicand| {
in_place_fix_variable(multiplicand, *r, prover_state.num_vars - prover_state.rou
});
```

```
}
```

변수 고정 과정:

- **첫 라운드 예외**: 첫 라운드에서는 고정할 변수가 없음
- **병렬 처리**: Rayon을 사용한 병렬 변수 고정 (성능 최적화)
- **In-place 연산**: 메모리 효율성을 위한 제자리 변수 고정

일변수 다항식 계산

```
let degree = prover_state.max_multiplicands; // the degree of univariate polynom
let round_length = 1usize << (prover_state.num_vars - prover_state.round);

// The pseudocode of what this is trying to do is:

// foreach t in 0..=degree compute
// sum over row in 0..round_length:
// sum over product in list_of_products:
// product over multiplicand in product:
// table = the mle of the multiplicand
// table[2b] * (1-t) + table[2b+1] * t
// This gives a vector of length degree + 1
```

계산 구조:

1. **외부 루프**: 곱셈 항들에 대한 반복

2. **중간 루프**: 하이퍼큐브의 행들에 대한 반복

3. **내부 루프**: 각 곱셈 항의 인수들에 대한 반복

효율적인 선형 보간

```
// This third+final loop give an efficient way of computing

// products[t] *= table[b << 1] * (S::one() - t_as_field) + table[(b << 1) + 1]

// It requires only 1 addition (plus the cumulative multiplication) to accomplis

// It relies on the fact that

// table[b << 1] * (S::one() - t_as_field) + table[(b << 1) + 1] * t_as_field ==

let mut start = table[b << 1];

let step = table[(b << 1) + 1] - start;


// The innermost loop loops over the values (t) that we are evaluating at.

products.iter_mut().take(degree).for_each(|product| {

*product *= start;

start += step;

});

products[degree] *= start;
```

최적화 기법:

- **선형 보간 최적화**: `f(0)(1-t) + f(1)t = f(0) + t(f(1)-f(0))` 공식 활용
- **단일 덧셈**: 각 평가 지점마다 하나의 덧셈만 필요
- **누적 곱셈**: 메모리 접근 최소화

변수 고정 함수

```
fn in_place_fix_variable<S: Scalar>(multiplicand: &mut [S], r_as_field: S, num_v

assert!(num_vars > 0, "invalid size of partial point");

for b in 0..(1 << num_vars) {

let left: S = multiplicand[b << 1];

let right: S = multiplicand[(b << 1) + 1];

multiplicand[b] = left + r_as_field * (right - left);

}

}
```

변수 고정 과정:

- **하이퍼큐브 축약**: 2^n 크기의 배열을 2^{n-1} 크기로 축약
- **선형 보간**: `f(r) = f(0) + r(f(1) - f(0))` 공식 사용
- **비트 연산**: 효율적인 인덱스 계산을 위한 비트 시프트 활용

성능 최적화

병렬 처리

```
let sums_iter = if_rayon!(

prover_state.list_of_products.par_iter(),

prover_state.list_of_products.iter()
```

)

병렬화 전략:

- **조건부 병렬화**: `rayon` 기능이 활성화된 경우에만 병렬 처리
- **데이터 병렬성**: 독립적인 곱셈 항들을 병렬로 처리
- **메모리 지역성**: 캐시 효율성을 고려한 데이터 접근 패턴

메모리 최적화

- **제자리 연산**: 추가 메모리 할당 없이 배열 수정
- **용량 예약**: `Vec::with_capacity` 를 통한 재할당 방지
- **메모리 로깅**: 성능 분석을 위한 메모리 사용량 추적

보안 특성

완전성 (Completeness)

정직한 증명자가 올바른 주장에 대해 항상 유효한 증명을 생성할 수 있습니다.

건전성 (Soundness)

거짓 주장에 대해서는 높은 확률로 검증이 실패합니다:

- **오류 확률**: 각 라운드마다 최대 `d/|F|` (d: 차수, |F|: 필드 크기)
- **전체 오류 확률**: $(d/|F|)^n$ (n: 변수 개수)

영지식성 (Zero-Knowledge)

시뮬레이터를 통해 실제 witness 없이도 구별 불가능한 증명을 생성할 수 있습니다.

실제 응용

SQL 쿼리 검증

1. **테이블 표현**: 데이터베이스 테이블을 다선형 확장으로 인코딩
2. **쿼리 변환**: SQL 연산을 다변수 다항식으로 변환
3. **결과 검증**: Sumcheck를 통해 쿼리 결과의 정확성 증명

복합 다항식 처리

```
// 예시: SELECT SUM(a * b) FROM table WHERE c > 0

// 이는 다음과 같은 다항식으로 표현됩니다:

// Σ_{x∈{0,1}ⁿ} a(x) * b(x) * filter(x)
```

이 Sumcheck 프로토콜 구현은 Proof of SQL의 핵심 증명 엔진으로, 효율적이고 안전한 영지식 증명을 제공합니다.